pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

LightGBM and Stacking

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Three boosters, then a stack

Three boosters

LightGBM

leaf-wise trees, bins

HistGradientBoosting

scikit-learn's own, bins

XGBoost

day 8, depth-limited trees

A stack

Base models give a probability for every account. A small model on top takes them as columns and learns how much to trust each.

The trap: which outputs the top model learns from.

Can several models that score about the same be combined into one that beats them all?

How LightGBM works

1. Bin the columns

at most 255 bins; splits among bin edges

2. Best leaf first

split the leaf, anywhere, with the largest gain

3. Until num_leaves

deep on one branch, shallow on another

4. Fit and add

gradients of the log loss, times the learning rate

You control

num_leaves, min_child_samples, learning_rate, n_estimators, and the sampling.

What comes out

An LGBMClassifier: predict_proba is the model probability of default in the file.

LightGBM: leaves, not depth, and bins

Depth-limited (day 8)

Splits every node at one level before going deeper. max_depth=4 allows at most 16 leaves.

Leaf-wise (LightGBM)

Splits whichever leaf gains most, anywhere, until num_leaves leaves (31 by default).

31

leaves in each of 300 trees

8 to 22

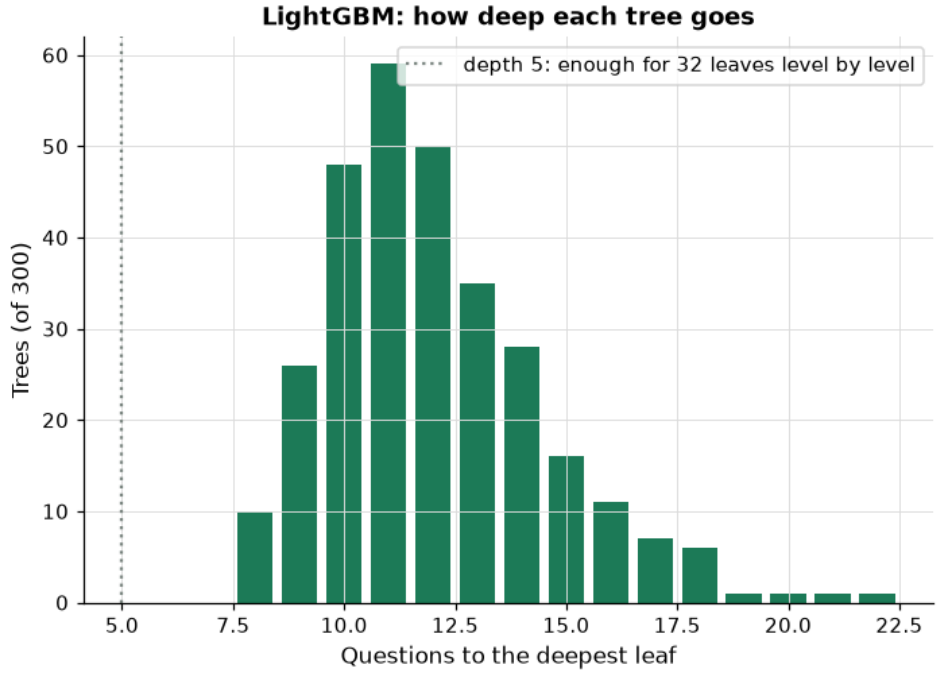
questions to the deepest leaf

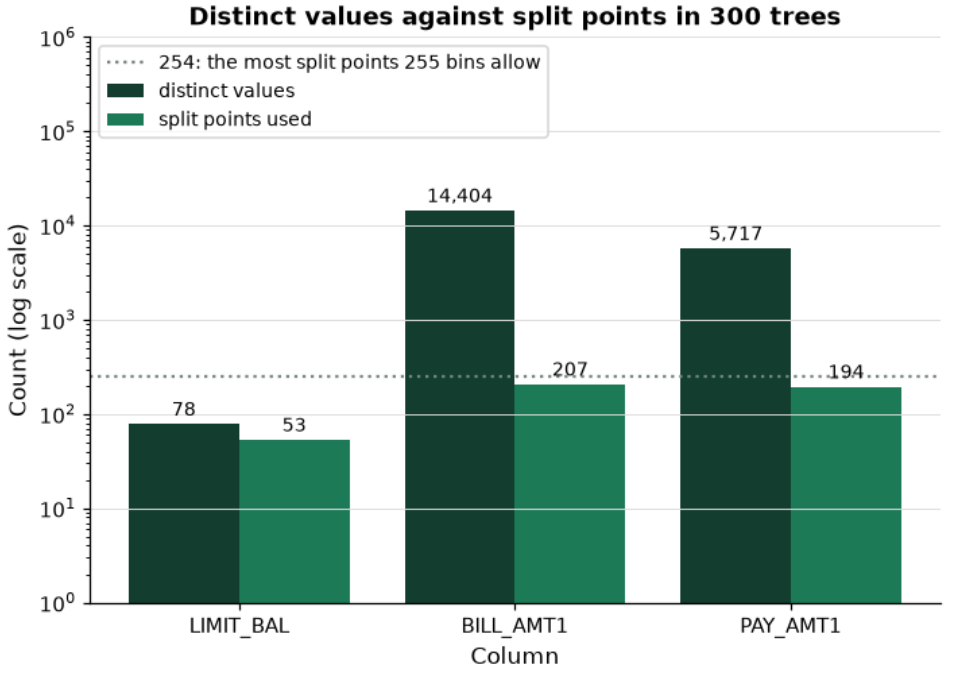
14,404 to 207

BILL_AMT1 values to split points

At most 255 bins a column (max_bin): never more than 254 split points to weigh.

Deep on a few branches, and binned





Every tree goes past depth 5 (8 to 22 questions). BILL_AMT1: 14,404 values, 207 split points.

LightGBM on the validation rows

0.7725

LightGBM, validation ROC AUC

0.7190

Course 4's logistic regression

0.9220

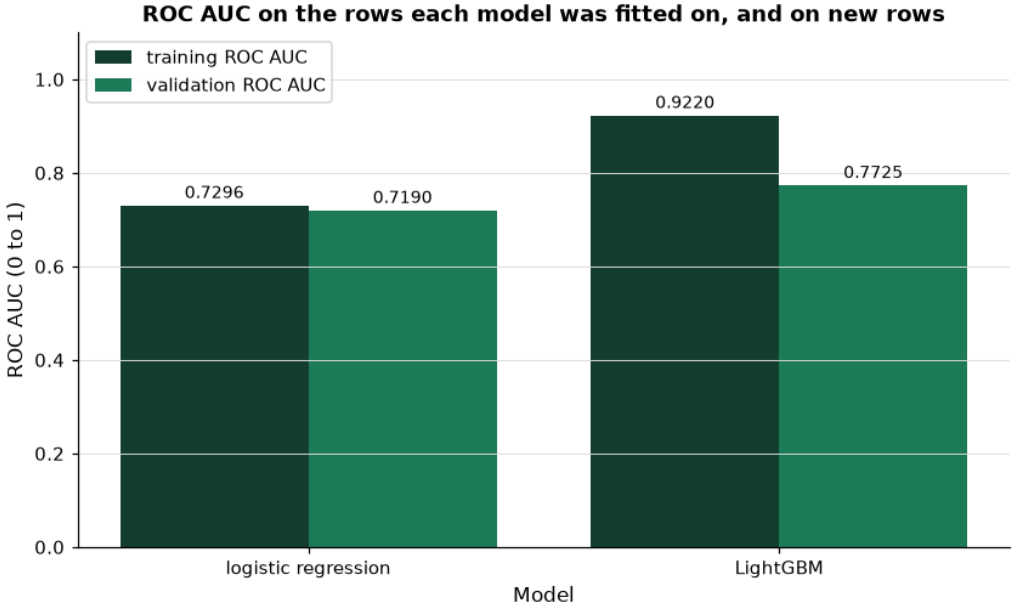
LightGBM on its own training rows

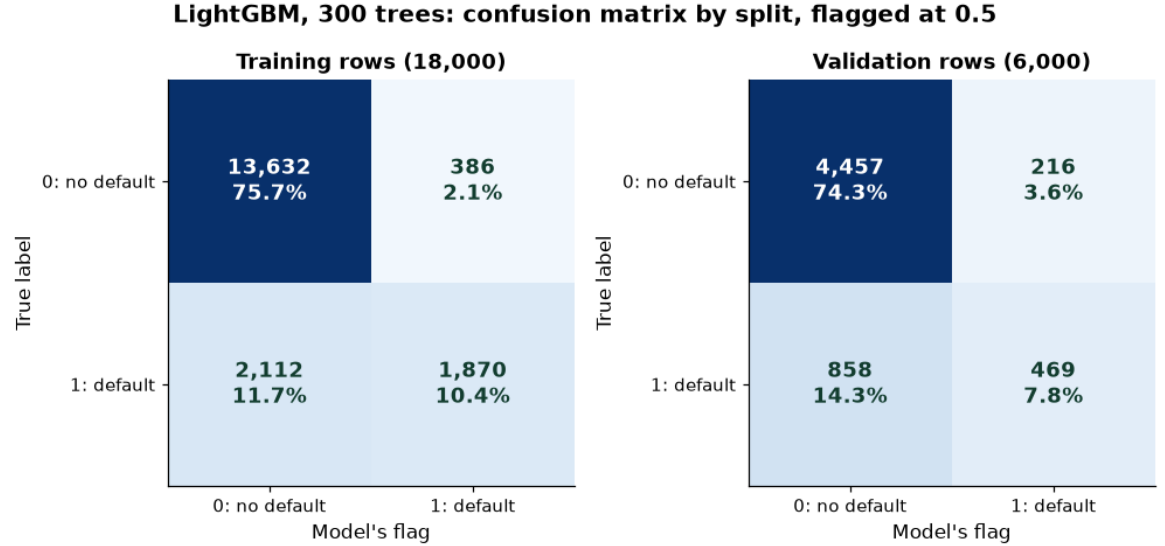
Average precision: LightGBM 0.5434, logistic regression 0.4946

```
LGBMClassifier(n_estimators=300, learning_rate=0.05, random_state=SEED, n_jobs=1, verbose=-1)
```

On rows it was fitted on, a model looks sharper than it is. Remember that for the stack.

Rows it has seen, rows it has not





Training minus validation ROC AUC: logistic 0.0106, LightGBM 0.1496. The training panel is the number a stack must never learn from.

Where the name comes from

Ke, Meng, Finley, Wang, Chen, Ma, Ye, Liu, 2017

"LightGBM: A Highly Efficient Gradient Boosting Decision Tree"

Advances in Neural Information Processing Systems 30 (NIPS 2017)

The same boosting as day 7, with trees grown and split in a faster way.

Source, read 5 October 2026: the paper's page in the NeurIPS proceedings (papers.nips.cc). Title, authors, volume, and year only; the paper itself was not read for this course.

How five-fold scoring works

1. Cut five folds

of the 18,000 training rows

2. Fit on four

score the fifth

3. Average

the five ROC AUCs

4. Read the spread

the noise to compare differences against

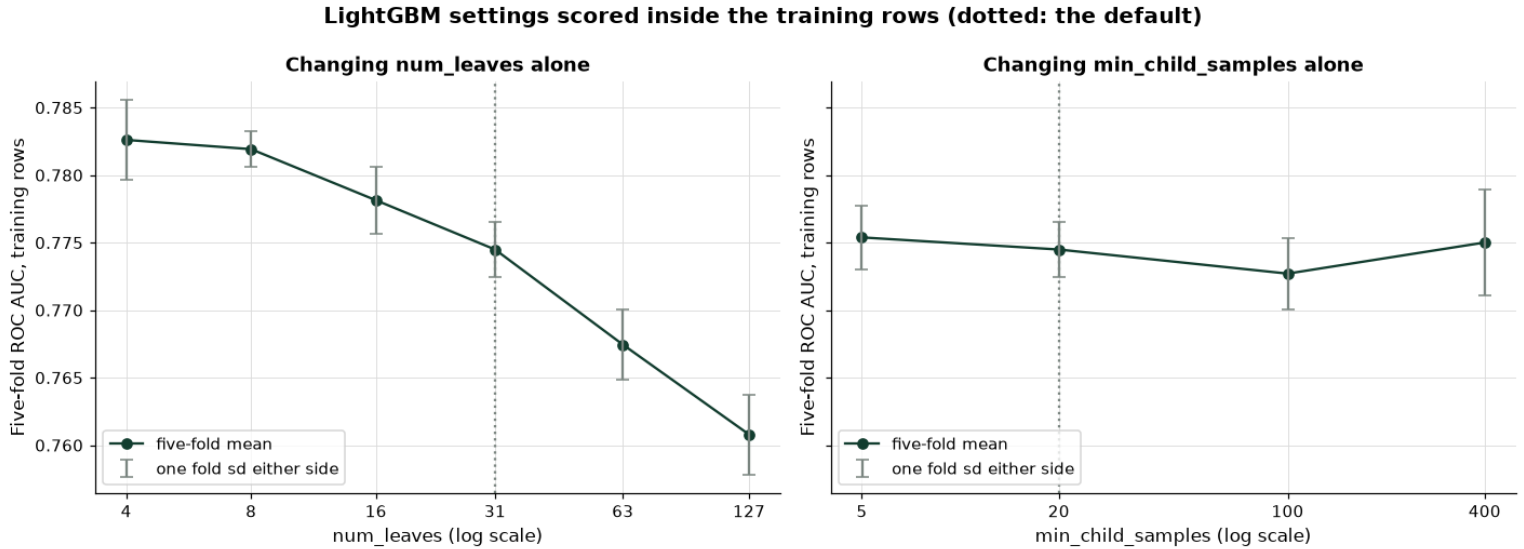
You control

The folds, the setting and its values, and the score.

What comes out

A mean and a standard deviation for each setting.

num_leaves and min_child_samples, inside the training rows



Five folds

Defaults: 0.7745 (sd 0.0020). Fewer leaves score higher; the leaf minimum stays within a bar.

Read every difference against the fold standard deviation. Nothing is chosen today.

How HistGradientBoosting works

1. Bin the columns

at most 255 bins

2. Grow by leaves

up to max_leaf_nodes, best split first

3. Fit and add

gradients of the log loss

4. What comes with it

no extra package, missing values, automatic early stopping

You control

max_iter, learning_rate, max_leaf_nodes, min_samples_leaf, l2_regularization.

What comes out

A model whose predict_proba is the model probability of default in the file.

HistGradientBoosting: scikit-learn's own

255

bins (max_bins)

31

leaves (max_leaf_nodes)

55

trees fitted of 100

0.7773

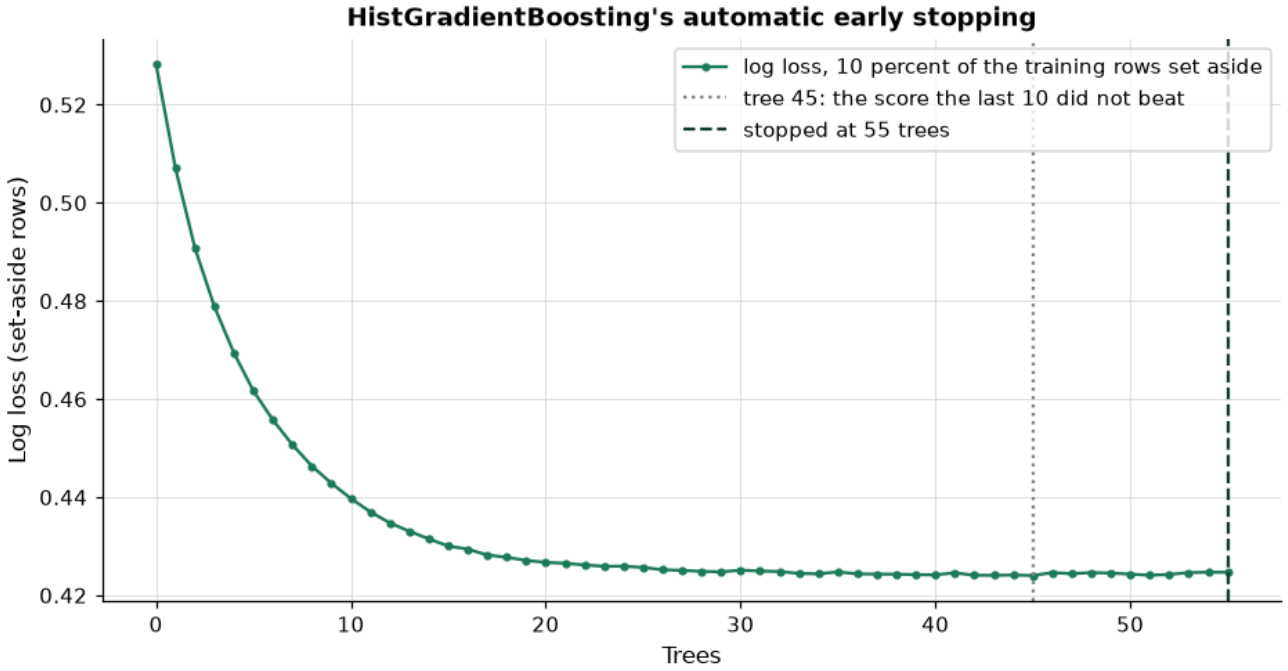
validation ROC AUC

early_stopping="auto"

On above 10,000 training rows: it sets aside 10 percent of the rows it is given (validation_fraction), picked with random_state=SEED, and stops when they stop improving.

The rows it stops on come from the training rows: day 8's rule, kept by the library.

Where it stopped



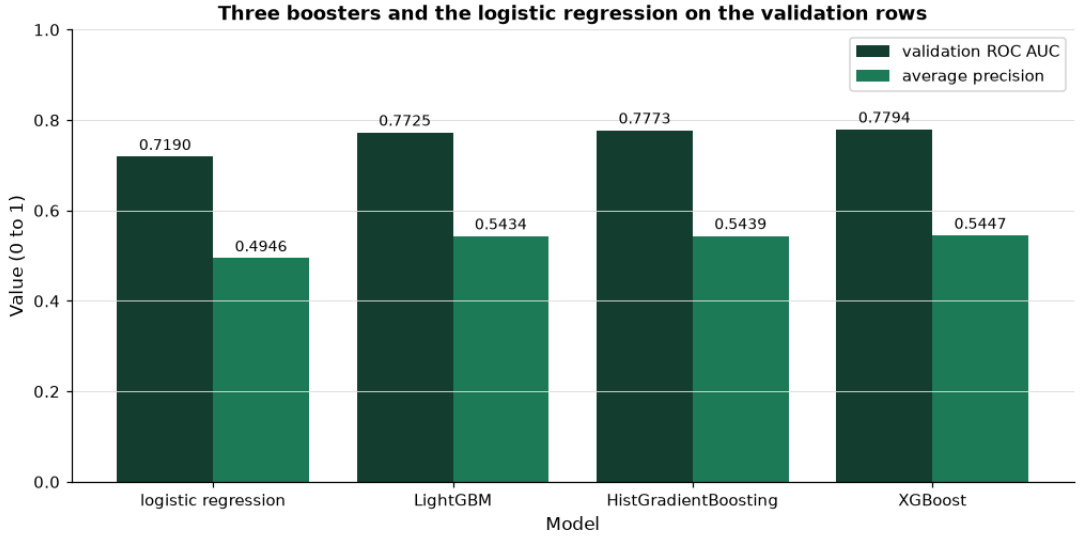
The rule

Stop when 10 trees in a row do not beat the score 10 trees back. Stopped at 55 trees.

Day 8 drew this by hand for XGBoost; here the library carves its stopping rows from the training rows.

Three boosters, one validation set

Model	ROC AUC	Avg precision
logistic regression	0.7190	0.4946
LightGBM	0.7725	0.5434
HistGradientBoosting	0.7773	0.5439
XGBoost	0.7794	0.5447



0.0069

spread among the boosters

Different trees, settings, and code, within a few thousandths. Day 10 measures the noise.

How stacking works

1. Out-of-fold columns

one probability per base model

2. Final model

a logistic regression on those columns

3. Refit the members

on all the training rows

4. Score new accounts

members first, then the final model

You control

The members, the final model, the folds, and passthrough.

What comes out

One fitted model whose predict_proba is the model probability of default in the file.

Stacking on out-of-fold outputs

1. Five folds

Fit each base model on four fifths of the training rows

2. Out of fold

Keep its output on the fifth it did not see

3. Two columns

logistic and lightgbm, correlation 0.8163

4. Final model

A logistic regression on the two columns

0.7743

LightGBM out of fold

0.9220

LightGBM in sample

4.0365

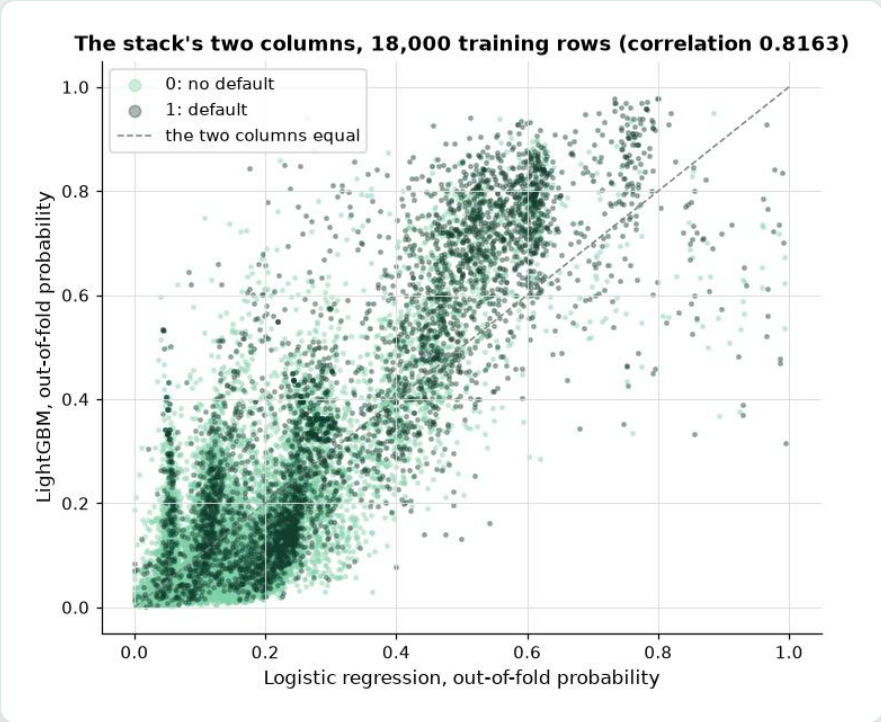
coefficient, lightgbm

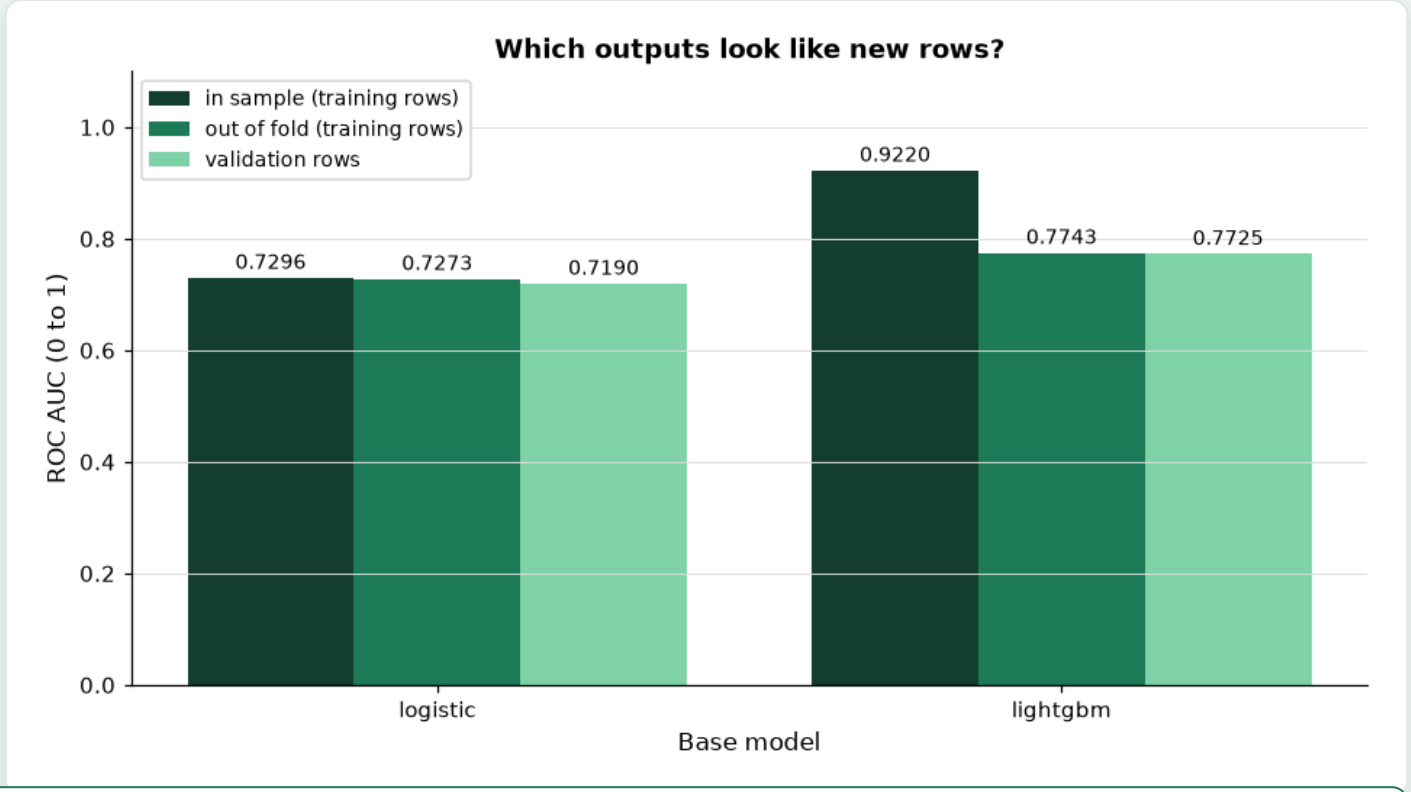
1.1551

coefficient, logistic

```
inputs = ensemblekit.stack_inputs(base_models, X_train, y_train)
```

The two columns, and which outputs look like new rows





LightGBM: 0.9220 in sample, 0.7743 out of fold, 0.7725 on the validation rows. The layer learns from the middle bar.

StackingClassifier does the same

```
StackingClassifier(list(base_models.items()), final_estimator=LogisticRegression(),
                   stack_method="predict_proba", cv=mlkit.cv_folds(), n_jobs=1)
```

Final model's coefficients

equal within 1e-10: True

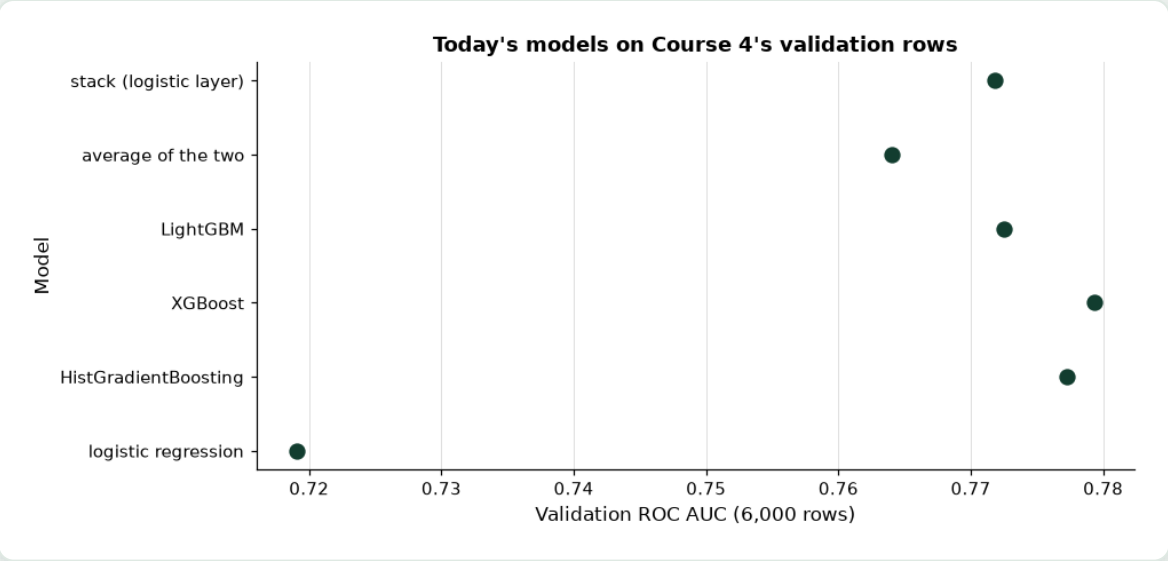
Validation outputs

equal within 1e-12: True

cv= is always a seeded splitter, never a bare number. The base models are then refitted on all the training rows.

Does the stack beat its best member?

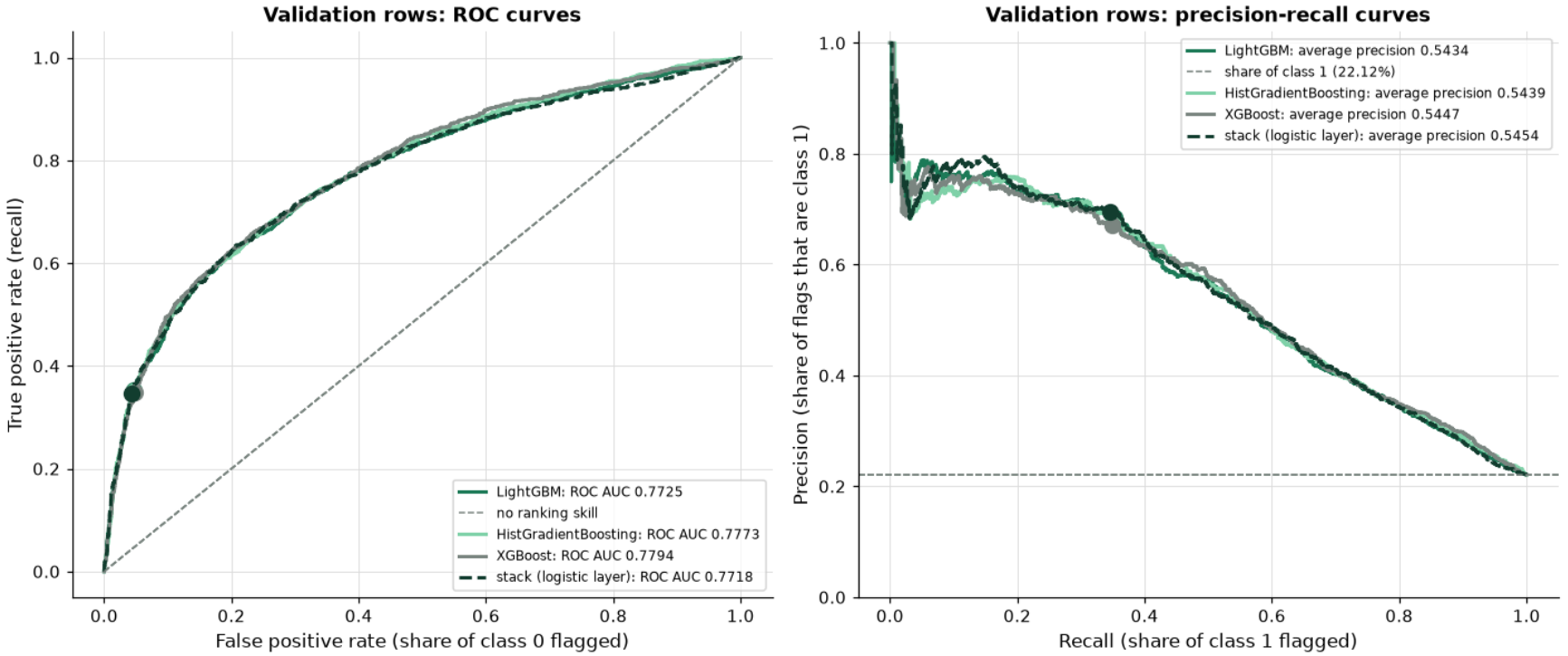
Model	ROC AUC	Avg precision
logistic regression	0.7190	0.4946
LightGBM	0.7725	0.5434
average of the two	0.7641	0.5354
stack (logistic layer)	0.7718	0.5454



Stack minus LightGBM: ROC AUC -0.0007, average precision +0.0020.

Not on ROC AUC (-0.0007); on average precision, yes (+0.0020). Both smaller than the boosters' spread of 0.0069.

Three boosters and the stack, as curves



Four curves almost on one another; the dashed stack follows LightGBM, the member its layer leans on.

Every setting of LGBMClassifier

Setting	Default	Tune first?
learning_rate	0.1	tune first
n_estimators	100	tune first
num_leaves	31	tune first
max_depth	-1	
min_child_samples	20	tune first
reg_lambda	0.0	tune first
subsample	1.0	tune first
subsample_freq	0	
colsample_bytree	1.0	tune first

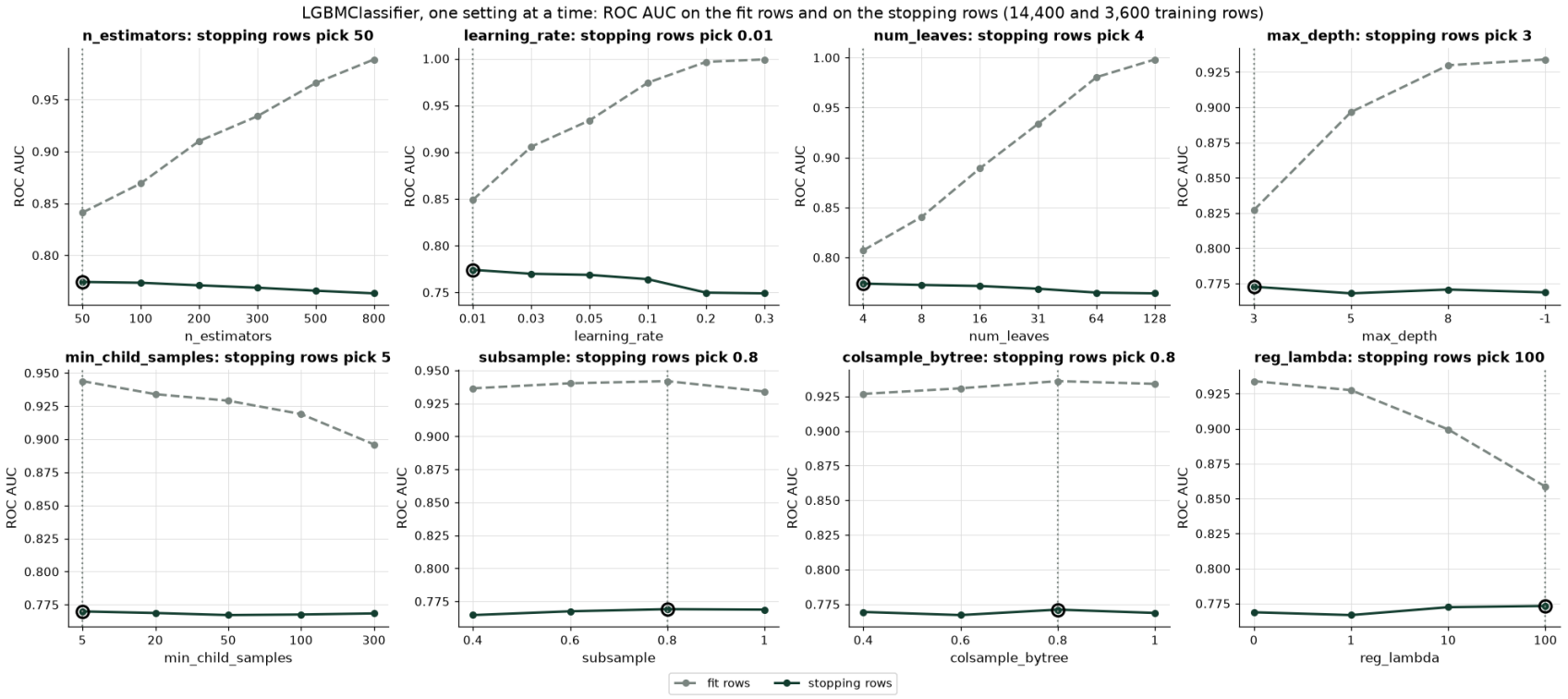
A trap in the table

subsample is 1.0 and subsample_freq is 0: bagging is off, so subsample=0.5 alone changes nothing. The notebook prints the proof.

Read it by group

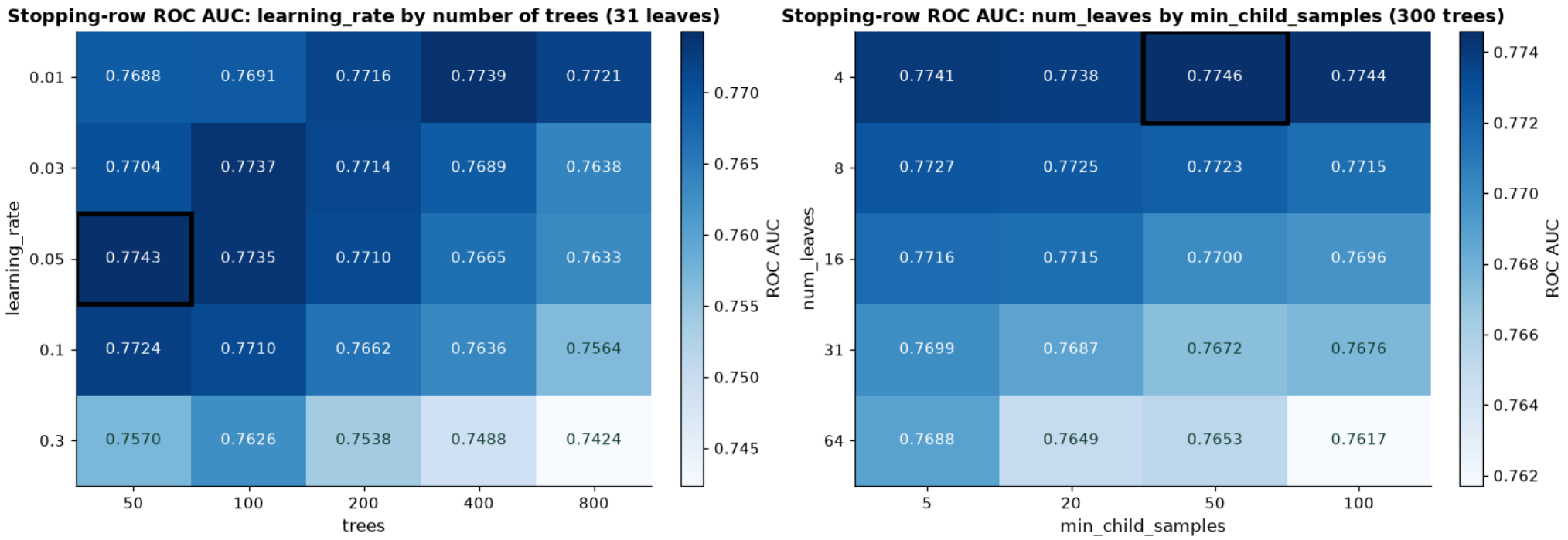
num_leaves is the main size control. min_split_gain, reg_alpha, and reg_lambda are day 8's penalties under other names. 20 settings, nearly all with a numeric default.

One LightGBM setting at a time



The fit rows move a lot, the stopping rows a few thousandths. Small trees are best on this file: the default of 31 leaves is past the peak.

LightGBM: rate against trees, and leaves



A smaller rate peaks later: they are one choice. Fewer leaves beat more; min_child_samples matters far less.

How CatBoost works

1. Symmetric trees

one question per level: depth 6 is 64 cells

2. Ordered encoding

a category becomes a target statistic from earlier rows

3. Ordered boosting

the same idea applied to the gradients

4. Four settings

random_seed, thread_count=1, verbose=0, allow_writing_files=False

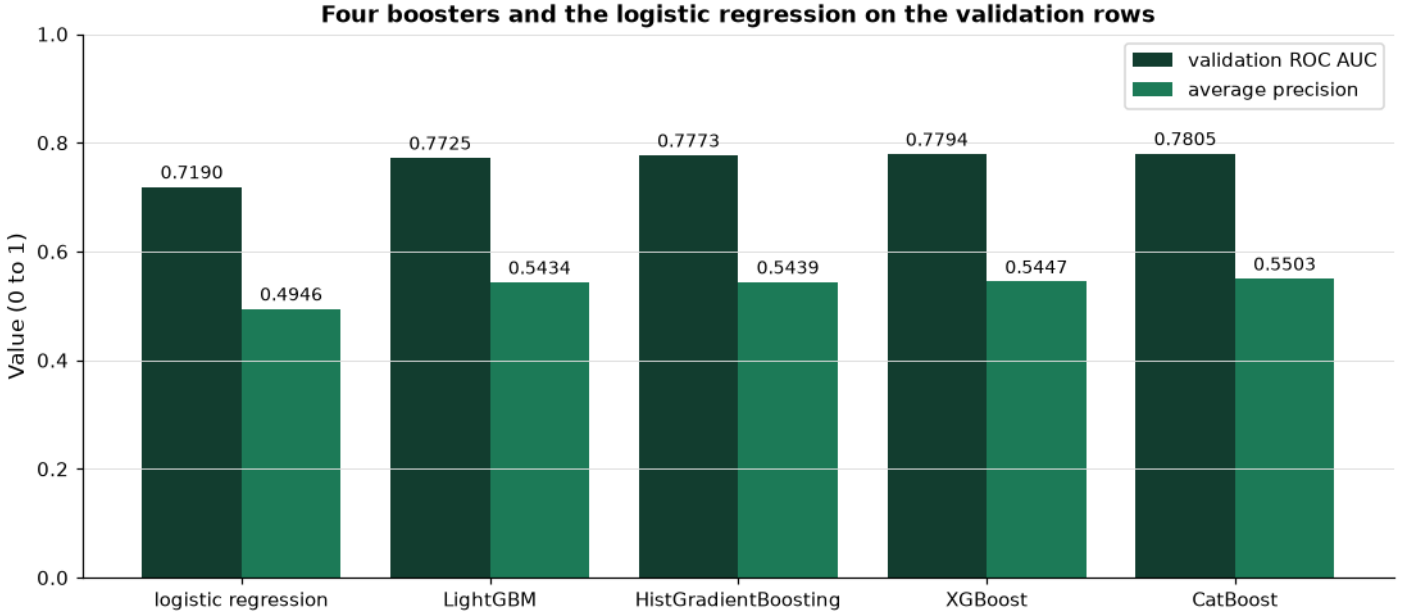
You control

iterations, learning_rate, depth, l2_leaf_reg, and which columns are categories.

What comes out

A CatBoostClassifier or CatBoostRegressor with the usual fit and predict. catboost==1.2.10.

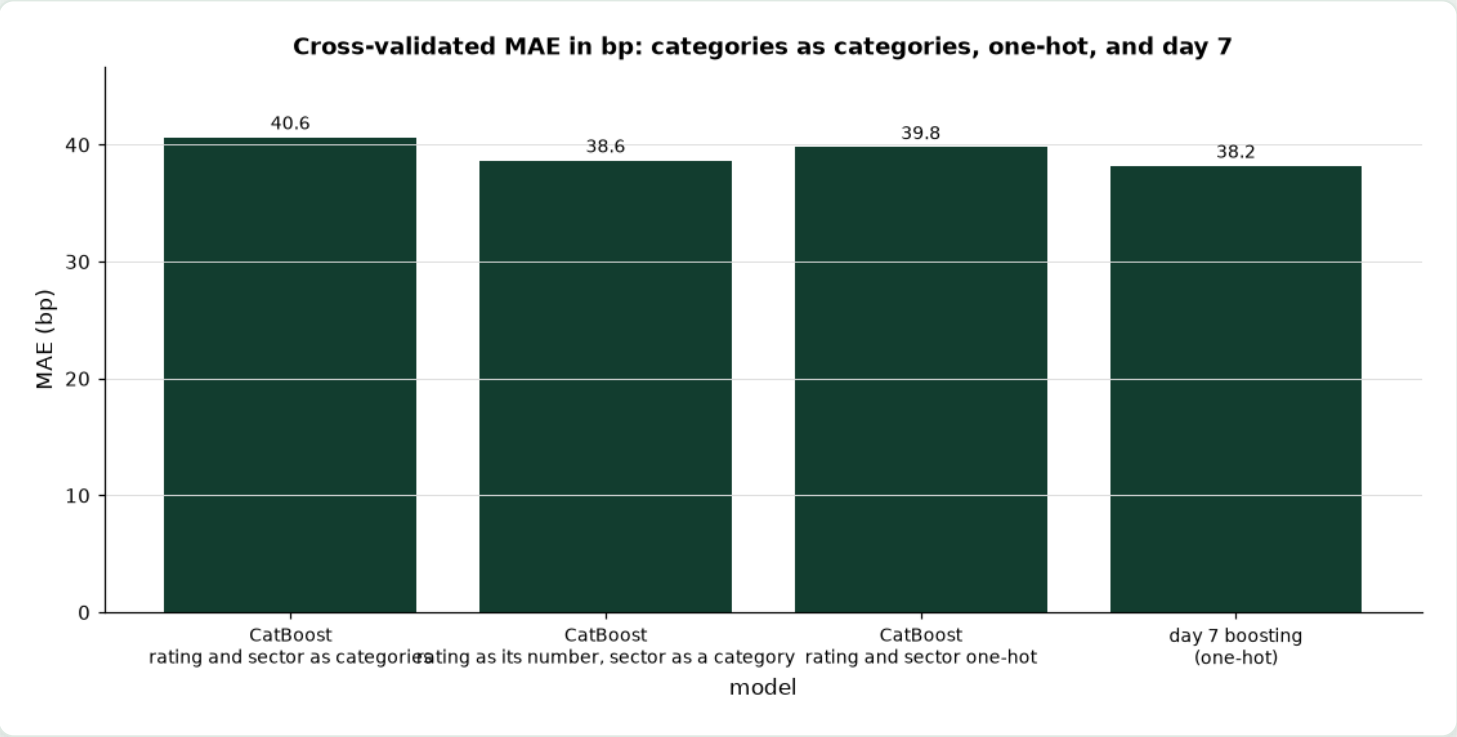
CatBoost beside the three boosters



Validation rows	ROC AUC	AP
logistic regression	0.7190	0.4946
LightGBM	0.7725	0.5434
HistGradientBoosting	0.7773	0.5439
XGBoost	0.7794	0.5447
CatBoost	0.7805	0.5503

Four libraries, a few thousandths apart, none tuned for this file. The card file has no categories: the bond file does.

Categories as categories, on bonds



Five folds, 615 bonds	MAE (bp)
CB: both as categories	40.6
CB: rating as its number	38.6
CB: both one-hot	39.8
day 7 boosting	38.2

Within a few basis points: the file has few categories. The rating as its ordered number beats the rating as a category.

Every setting of CatBoostClassifier

Setting	Passed	In force	Tune first?
iterations	300	300	tune first
learning_rate	0.05	0.05	tune first
depth	-	6	tune first
l2_leaf_reg	-	3	tune first
random_strength	-	1	
rsm	-	1	
bootstrap_type	-	MVS	
boosting_type	-	Plain	
grow_policy	-	SymmetricTree	

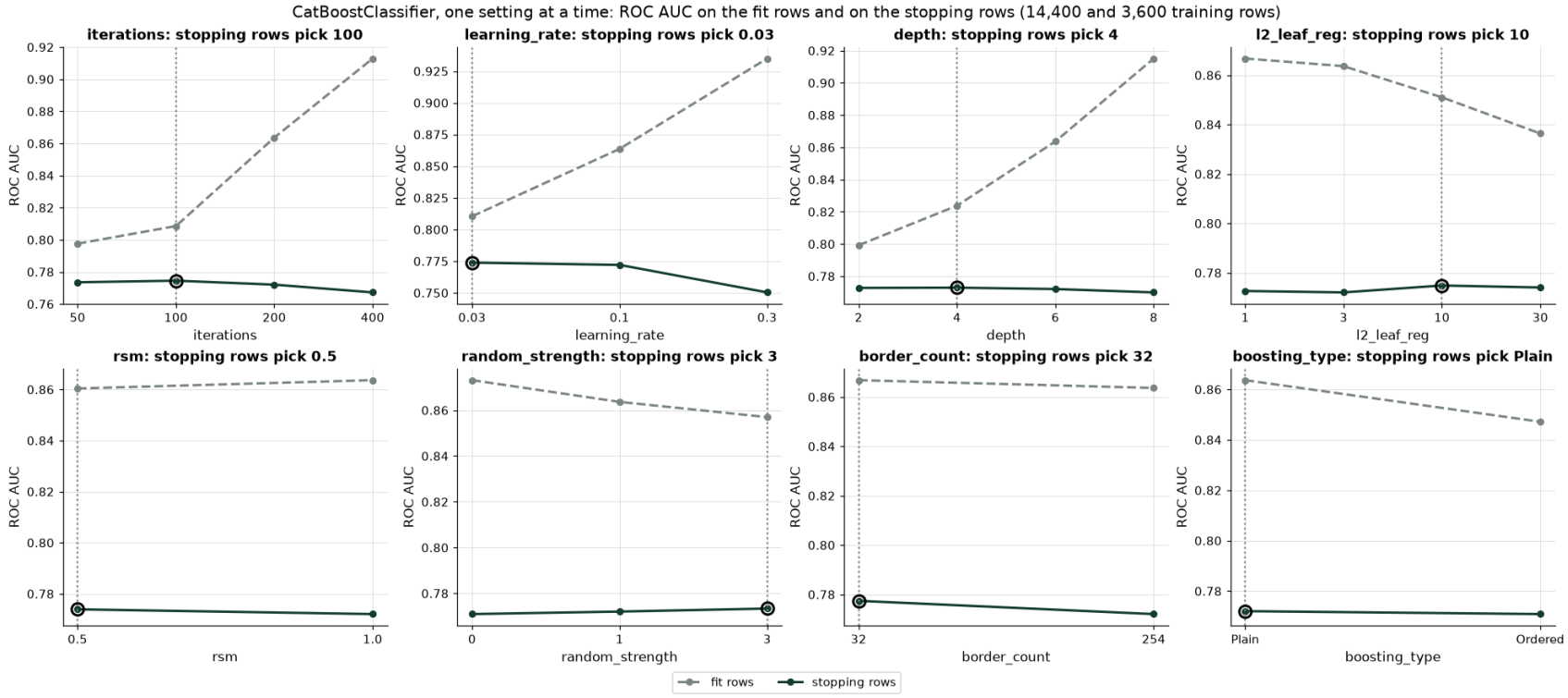
Read from the fitted model

get_params() shows only what you passed. get_all_params() gives the 41 settings in force, with the values CatBoost chose: a learning rate picked from the data size, symmetric trees, plain boosting.

Tune first

iterations with learning_rate, depth, and l2_leaf_reg. Most of the 41 rows are housekeeping the course does not use.

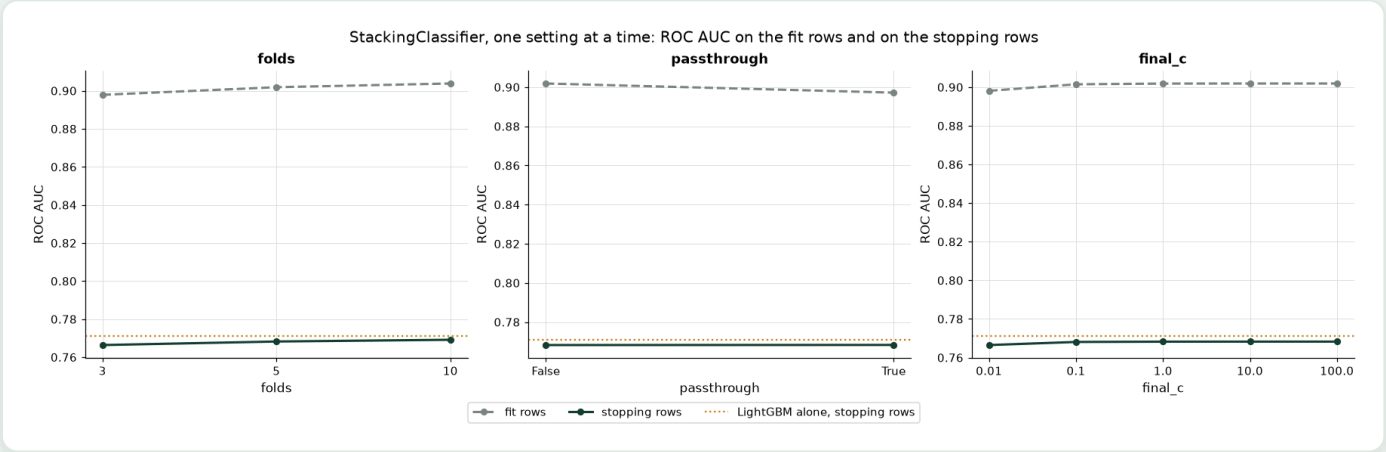
One CatBoost setting at a time



The fit rows move a lot, the stopping rows a few thousandths. Symmetric trees overfit later, not never. The panels describe; they do not choose.

Every setting of StackingClassifier

Setting	Default	Tune first?
estimators	none, required	tune first
final_estimator	None	tune first
cv	None	tune first
stack_method	'auto'	
n_jobs	None	
passthrough	False	tune first
verbose	0	



cv and passthrough decide what the final layer learns from. Against the member alone the stack gains little, as 9.8 found.

Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(B2,C2)`	the plain blend: 0.2409 on the first account
`=$H$6*B2+(1-$H$6)*C2`	a weighted blend, weight 0.3 in H6
`=1/(1+EXP(-($H$2+$H$3*B2+$H$4*C2)))`	the stack's output: 0.2034 on the first account
`=AVERAGE(E2:E6001)`	average of the stack's outputs: 0.2190

No LightGBM, no boosting, no cross-validation in Excel. Intercept -2.5861 and coefficients come from Python; Excel applies them.

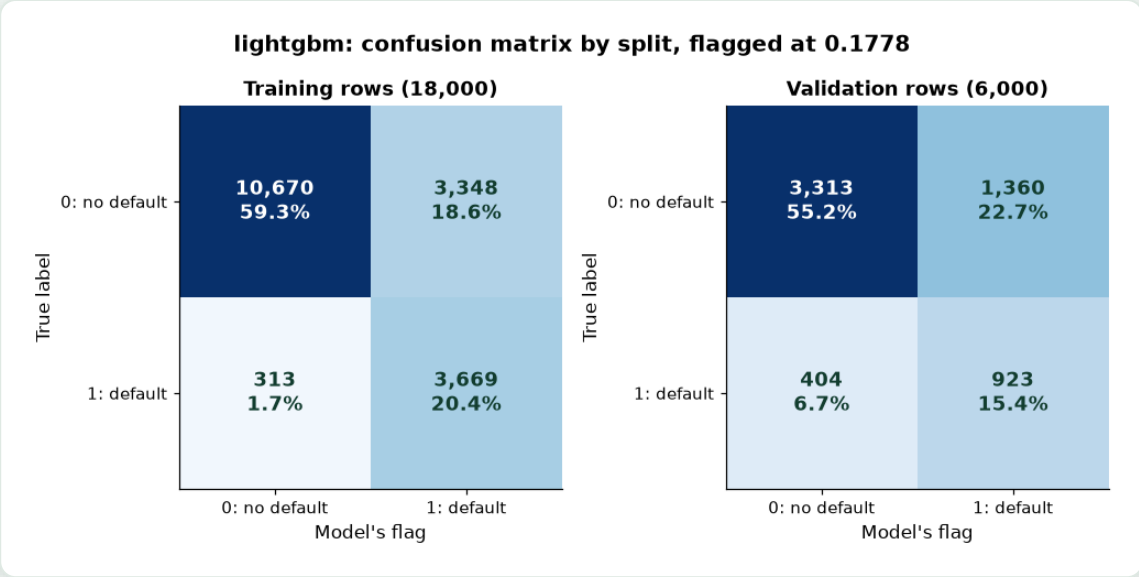
Save the day: the log and git blame

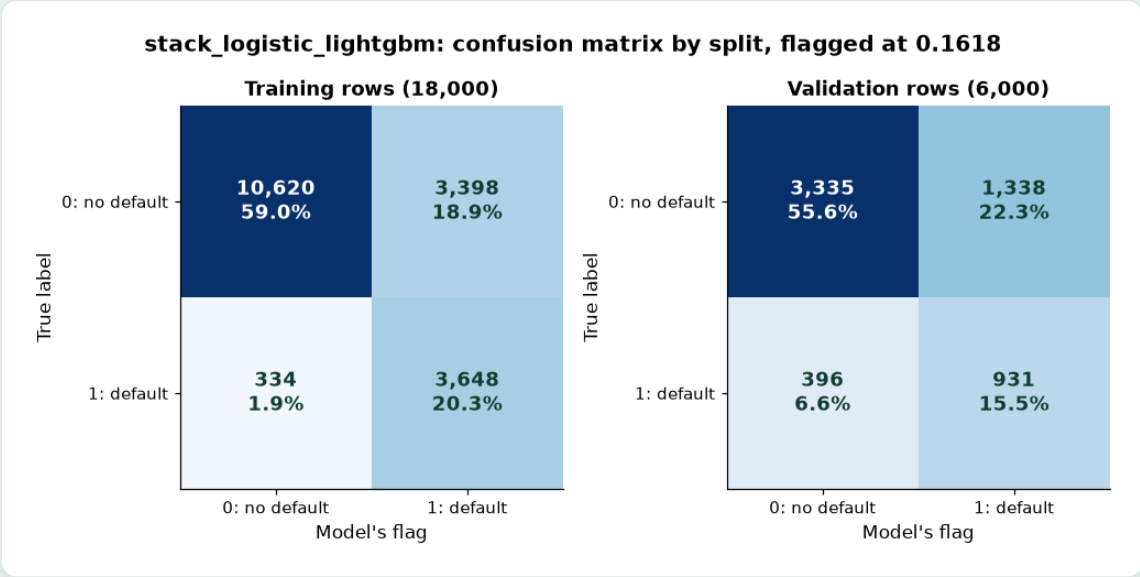
```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
lightgbm,validation,0.7725,0.5434,0.1778,0.7060,0.4043,0.6956,0.5114
xgboost,validation,0.7794,0.5447,0.1812,0.7133,0.4115,0.6888,0.5152
blend_logistic_lightgbm,validation,0.7641,0.5354,0.1853,0.6987,0.3976,0.7038,0.5082
stack_logistic_lightgbm,validation,0.7718,0.5454,0.1618,0.7110,0.4103,0.7016,0.5178
```

```
git blame -s --root -L "/def stack_inputs/,+1" ensemblekit.py
```

Thresholds from out-of-fold probabilities on the training rows. All-zero baseline 0.7788, never to choose. 35 tests pass.

LightGBM and the stack by split, at their own thresholds





Validation recall 0.6888 to 0.7038 across the log, near the 0.70 aimed for. The training panels overshoot: rows the base models saw.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the stack's columns are in-sample outputs: each base model scores the rows it was fitted on.

Code that runs is not the same as code that is right.

What you can do now

- LightGBM grows leaf-wise to num_leaves, on bins
- n_jobs=1 and verbose=-1 on every LightGBM
- Settings compared by folds of the training rows
- A stack's columns are out-of-fold outputs
- Say plainly when a stack does not beat its best member

Week 2, Day 5 Case study: boosting on the card file, and the noise in a gap